

Lab-02: 2D object Coloring with OpenGL

Le Quoc Van - 22125119

March 12, 2025



1 Introduction

This report presents the implementation details of a computer graphics project using OpenGL and GLUT. The goal is to create and test various graphical objects, such as circles, rectangles, hexagons, and ellipses, within an interactive window.

1.1 Demonstrate video

[Demonstrate link](#)

2 Project Structure

The project is structured as follows:

- **core/** - Contains core implementation files.
- **include/** - Header files for different graphical objects.
- **tests/** - Test files for verifying individual features.
- **CMakeLists.txt** - Build configuration file.
- **build.sh** - Shell script for building and running the project.

3 Implementation Details

3.1 Window Class

The Window class initializes an OpenGL window and handles user interactions.

```

1 class Window {
2 private:
3     std::string title;
4     int width, height;
5     bool fullscreen;
6     static void displayCallback();
7     static void keyCallback(unsigned char key, int x, int y);
8     static void mouseCallback(int button, int state, int x, int y);
9 public:
10    Window(std::string title, int width, int height);
11    void run();
12 };

```

Algorithm 1: Window Class Definition

3.2 Shape Implementations

The following shapes are implemented:

- **Circle** - Drawn using a series of line segments.

March 12, 2025

- **Ellipse** - Similar to a circle but with two radii.
- **Rectangle** - Defined using two corner points.
- **Hexagon** - Constructed using six vertices.

3.2.1 Circle Implementation

```

1 class Circle : public Object {
2 private:
3     std::array<int, 2> center;
4     int radius;
5     int num_segments;
6     bool needBoundary;
7 public:
8     Circle(std::array<int, 2> center, int radius, std::array<unsigned char, 3>
9         boundary_color, bool needBoundary, int num_segments = 100);
10    void draw();
11 };

```

Algorithm 2: Circle Class Definition

3.2.2 Hexagon Implementation

```

1 class Hexagon : public Object {
2 private:
3     std::array<int, 2> center;
4     int radius;
5     std::vector<std::array<int, 2>> vertices;
6     void calculateVertices();
7 public:
8     Hexagon(std::array<int, 2> center, int radius, std::array<unsigned char, 3>
9         boundary_color, bool needBoundary);
10    void draw();
11 };

```

Algorithm 3: Hexagon Class Definition

3.3 Configuration Popup

A popup is implemented to allow users to choose shapes and colors interactively.

```

1 class ConfigPopup {
2 private:
3     std::vector<std::string> shapes = {"Circle", "Ellipse", "Rectangle", "Polygon"};
4     std::vector<std::string> colors = {"Black", "Red", "Green", "Blue"};
5     int selectedShape, selectedColor;
6 public:
7     ConfigPopup(int width, int height);
8     void display();
9     int getShape();
10    int getColor();

```

```
11 };
```

Algorithm 4: ConfigPopup Class Definition

4 Testing and Execution

The following test cases verify the implementation:

- **hello_world.cpp** - Tests basic GLUT functionality.
- **window.cpp** - Verifies the window rendering system.
- **circle.cpp** - Ensures circles are drawn correctly.
- **rectangle.cpp** - Tests rectangle rendering.
- **hexagon.cpp** - Validates hexagon generation.
- **ellipse.cpp** - Checks ellipse rendering.

To execute a test, use:

```
1 ./build.sh
2 ./build/<executable_name>
```

5 Conclusion

This project demonstrates basic 2D rendering using OpenGL and GLUT. Future work could include user interaction enhancements and additional shape transformations.